

动态规划

1. 0/1 背包问题

给定 n 个物品和容量为 C 的背包，物品 i 的重量为 w_i ，价值为 v_i 。每个物品只能选择“装入”或“不装入”（不可分割），要求在背包总重量不超过 C 的前提下，使装入物品的总价值最大。

2. 数字加减乘号最大结果

给定 N 个 1-9 的数字 $v_1, v_2, \dots, v_n$ ，不改变数字相对位置，在相邻数字间加入 K 个乘号和 $N - K - 1$ 个加号（每位置必加一个符号），括号可任意添加，使最终结果最大。

3. 序列划分最小化最大值和

给定长度为 N 的整数序列 $(a_1, a_2, \dots, a_n)$ ，将其划分为若干连续子序列，每个子序列的和 $\leq B$ 。求使所有子序列“最大值”之和最小的划分方案。

4. 树的节点着色（最大化黑色节点数）

对一棵以 r 为根的树进行节点着色，每个节点可着黑色或白色。约束：相邻节点不能着相同黑色（但可同白）。求使树中黑色节点数最多的着色方案。

5. 字符串分词（最大化质量和）

给定字符串 $y = y_1 y_2 \dots y_n$ ，将其切分为若干连续子串（每个子串为合法词汇）。函数 $quality(x)$ 返回词汇 x 的质量，求使总质量最大的分词方案。

6. 带权活动选择问题

给定 n 个活动，每个活动 $a_i = (s_i, f_i, v_i)$ ，其中 s_i 为开始时间， f_i 为结束时间， v_i 为权重。选择若干不重叠的活动，使权重和最大。

7. 最大子数组问题（分治法）

给定 n 个整数（有正有负）的数组 A ，设计 $O(n \log n)$ 算法，找出和最大的非空连续子数组。

8. 最长非降子序列（LIS）

给定长度为 n 的序列 A ，求最长非降子序列（LIS）的长度（非降指 $a_i \leq a_j$ 对 $i < j$ 成立，子序列不要求连续）。

9. 矩阵连乘问题

给定 n 个矩阵 $M_1, M_2, \dots, M_n$ ，其中 M_i 维数为 $r_{i-1} \times r_i$ 。矩阵乘法满足结合律，不同的括号化方式导致不同的乘法次数。求使乘法运算次数最少的最优括号化方案。

10. 钢条切割问题

给定长度为 n 英寸的钢条和价格表 p_i ($i = 1 \sim n$)，钢条可切割为任意段，求切割后总收益最大。例如： $n = 4$ ， $p = [1, 5, 8, 9]$ 时，最优方案为切成 2 段长度 2，收益 $5 + 5 = 10$ 。

11. 最长公共子序列（LCS）

给定两个序列 $X = (x_1, x_2, \dots, x_m)$ 和 $Y = (y_1, y_2, \dots, y_n)$ ，求其最长公共子序列的长度及具体序列（子序列不要求连续）。例如： $X = \text{"ABCB DAB"}$ ， $Y = \text{"BDCABA"}$ ，LCS 为 "BCAB" 或 "BDAB" ，长度为 4。

12. 最优二叉搜索树

给定 n 个有序关键字 $k_1 < k_2 < \dots < k_n$ ，每个关键字 k_i 的成功搜索概率为 p_i ，失败搜索概率为 $q_0, q_1, \dots, q_n$ 。构造使平均搜索长度最小的二叉搜索树。

13. DAG 最安全路径问题

给定有向无环图 $G = (V, E)$ ，边 e_i 的事故概率为 p_i 。路径安全性定义为路径上所有边概率的乘积。求从起点 s 到终点 t 的最安全路径（安全性最大的路径）。

14. 环形房间化学品存放问题

给定 n 个首尾相连的环形房间，每个房间容量为 w_i （杂物房间容量为 0）。约束：化学品不能存放在相邻房间。求最多能存放的化学品总容量。

15. 受限最短路径（带钱数约束）

给定无向图 $G = (V, E)$ ，边 e_{uv} 长度为 l_{uv} ，顶点 u 花费为 $c[u]$ 。从 s 出发到 t ，初始钱数为 M 。每经过顶点 u 需支付 $c[u]$ （剩余钱数 $\geq c[u]$ 才能通过），求 s 到 t 的最短路径长度。

16. 完全占满背包（恰好装满）

给定 n 个物品，物品 i 的大小为 w_i 、价值为 v_i ，背包容量为 C 。要求选择一组物品，恰好占满背包容量（选中物品大小和恰好为 C ），且总价值最大。

17. 数组最大子段的个数（O(n)）

设 $A[1..n]$ 是长度为 n 的数组，存放可重复的整数。最大子段指其和不少于任何子段和的子段。要求设计 $O(n)$ 算法，求最大子段的个数。

18. 最少插入字符使字符串成为回文

给定字符串 S ，设计算法在 S 中插入最少的字符，使其成为回文字符串。例如： $S = \text{"ab"}$ ，插入 1 个字符得 "aba" 或 "bab" 。

19. 二维矩阵的最大子矩阵和

给定 $m \times n$ 的二维整数矩阵 A （含正、负、零），找到和最大的子矩阵，并输出其和。

1. 0/1 背包问题

给定 n 个物品和容量为 C 的背包，物品 i 的重量为 w_i ，价值为 v_i 。每个物品只能选择”装入”或”不装入”（不可分割），要求在背包总重量不超过 C 的前提下，使装入物品的总价值最大。

- **状态定义：** $dp[i][j]$ 表示”前 i 个物品（物品 1 i ）放入容量为 j 的背包”的最大价值（子问题：限定物品范围和背包容量的最优解）
- **转移方程：**若 $j < w_i$ 则 $dp[i][j] = dp[i - 1][j]$ ；若 $j \geq w_i$ 则 $dp[i][j] = \max(dp[i - 1][j], dp[i - 1][j - w_i] + v_i)$
- **初始化：** $dp[0][j] = 0$ ($j=0 \sim C$)， $dp[i][0] = 0$ ($i=0 \sim n$)
- **计算顺序：** i 从 1 到 n ， j 从 1 到 C
- **输出结果：** $dp[n][C]$

```
// 二维DP版本
int knapsack(int n, int C, vector<int>& w, vector<int>& v) {
    vector<vector<int>>> dp(n + 1, vector<int>(C + 1, 0));

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= C; j++) {
            if (j < w[i-1]) {
                dp[i][j] = dp[i-1][j];
            } else {
                dp[i][j] = max(dp[i-1][j], dp[i-1][j-w[i-1]] + v[i-1]);
            }
        }
    }
    return dp[n][C];
}

// 空间优化版本（一维数组）
int knapsackOptimized(int n, int C, vector<int>& w, vector<int>& v) {
    vector<int> dp(C + 1, 0);

    for (int i = 0; i < n; i++) {
        for (int j = C; j >= w[i]; j--) { // 逆序遍历！
            dp[j] = max(dp[j], dp[j-w[i]] + v[i]);
        }
    }
    return dp[C];
}
```

1. 建二维表格，行表示依次考虑每个物品，列表示背包容量从 0 到 C
2. 从第一行第一列开始逐格填充：若当前物品重量超过当前容量，直接抄写上一行同列的值；若能装下，则比较”不装它”和”装它”两个值，取较大者
3. 空间优化可用一维数组，容量必须从大到小遍历
4. 最终右下角格子的值即为答案

2. 数字加减乘号最大结果

给定 N 个 1-9 的数字 $v_1, v_2, \dots, v_n$ ，不改变数字相对位置，在相邻数字间加入 K 个乘号和 $N - K - 1$ 个加号（每位置必加一个符号），括号可任意添加，使最终结果最大。

- **状态定义：** $dp[i][k]$ 表示前 i 个数字用 k 个乘号的最大结果
- **转移方程：**

$$dp[i][k] = \max_{k \leq j < i} \{ dp[j][k - 1] \times (sum[i] - sum[j]) \}$$

其中 $sum[i]$ 为前 i 个数字的和

- **边界条件：** $dp[i][0] = sum[i]$ (0 个乘号即全为加号)
- **计算顺序：** k 从 1 到 K ， i 从 $k + 1$ 到 n
- **时间复杂度：** $O(K \cdot n^2)$

```
int maxResultWithOperators(vector<int>& v, int K) {
    int n = v.size();

    // 预处理前缀和
    vector<int> sum(n + 1, 0);
    for (int i = 1; i <= n; i++) {
        sum[i] = sum[i-1] + v[i-1];
    }

    // dp[i][k]: 前i个数字用k个乘号的最大结果
    vector<vector<long long>>> dp(n + 1, vector<long long>(K + 1, 0));

    // 初始化: k=0时全为加号
    for (int i = 1; i <= n; i++) {
        dp[i][0] = sum[i];
    }

    // 填充DP表
    for (int k = 1; k <= K; k++) {
        for (int i = k + 1; i <= n; i++) {
            for (int j = k; j < i; j++) {
                long long current = dp[j][k-1] * (sum[i] - sum[j]);
                dp[i][k] = max(dp[i][k], current);
            }
        }
    }

    return dp[n][K];
}
```

1. 先算前缀和数组，方便快速求任意连续一段数字的和
2. 建二维表，行表示数字个数，列表示乘号个数
3. 枚举最后一个乘号位置，把结果分为”前面那段的 $k-1$ 个乘号的最大值”乘以”后面那段连续数字的和”
4. 表格右下角即为最终答案

3. 序列划分最小化最大值和

给定长度为 N 的整数序列 $(a_1, a_2, \dots, a_n)$ ，将其划分为若干连续子序列，每个子序列的和 $\leq B$ 。求使所有子序列”最大值”之和最小的划分方案。

- 状态定义： $dp[i]$ 表示前 i 个元素的最小最大值和
- 转移方程：

$$dp[i] = \min_{0 \leq j < i, \text{sum}(a_{j+1..i}) \leq B} \{dp[j] + \max(a_{j+1..i})\}$$

- 优化技巧：前缀和 $O(1)$ 计算子序列和，反向遍历 j 时动态更新 max 值

```
int minMaxSumPartition(vector<int>& a, int B) {
    int n = a.size();

    // 前缀和
    vector<int> sum(n + 1, 0);
    for (int i = 1; i <= n; i++) {
        sum[i] = sum[i-1] + a[i-1];
    }

    // DP初始化
    const int INF = 1e9;
    vector<int> dp(n + 1, INF);
    dp[0] = 0;

    for (int i = 1; i <= n; i++) {
        int currentMax = 0;

        // 反向遍历，提前终止
        for (int j = i - 1; j >= 0; j--) {
            currentMax = max(currentMax, a[j]);
            int subSum = sum[i] - sum[j];

            if (subSum > B) break; // 提前终止

            dp[i] = min(dp[i], dp[j] + currentMax);
        }
    }

    return dp[n];
}
```

1. 建一维数组， $dp[i]$ 表示前 i 个元素划分后的最小代价
2. 从前往后计算：尝试让最后一段从 $j+1$ 到 i
3. 只要和不超过 B ，就比较” $dp[j]$ 加这段最大值”
4. 一旦超过 B 就停止左移

4. 树的节点着色（最大化黑色节点数）

对一棵以 r 为根的树进行节点着色，每个节点可着黑色或白色。约束：相邻节点不能着相同黑色（但可同白）。求使树中黑色节点数最多的着色方案。

- 状态定义： $dp[u][0]$ 表示节点 u 着白色时子树最大黑色节点数， $dp[u][1]$ 表示节点 u 着黑色时子树最大黑色节点数
- 转移方程： $dp[u][1] = 1 + \sum_{v \in children(u)} dp[v][0]$, $dp[u][0] = \sum_{v \in children(u)} \max(dp[v][0], dp[v][1])$
- 遍历顺序：后序遍历（先子节点后父节点）
- 最终结果： $\max(dp[r][0], dp[r][1])$

```
struct TreeNode {
    vector<TreeNode*> children;
};

pair<int, int> treeColoring(TreeNode* u) {
    // 返回 {着白时的最大黑数, 着黑时的最大黑数}
    int white = 0, black = 1; // 初始化：无子节点时

    for (TreeNode* v : u->children) {
        auto [w, b] = treeColoring(v);

        // u着黑：子节点必须着白
        black += w;

        // u着白：子节点可选最大
        white += max(w, b);
    }

    return {white, black};
}

int maxBlackNodes(TreeNode* root) {
    auto [white, black] = treeColoring(root);
    return max(white, black);
}
```

1. 从根节点开始后序遍历（先处理完所有孩子，再处理自己）
2. 对每个节点算两个值：染白时子树最多几个黑节点、染黑时子树最多几个黑节点
3. 若染黑，所有孩子只能染白；若染白，每个孩子自由选择
4. 根节点取两者较大值

5. 字符串分词（最大化质量和）

给定字符串 $y = y_1y_2 \dots y_n$ ，将其切分为若干连续子串（每个子串为合法词汇）。函数 $quality(x)$ 返回词汇 x 的质量，求使总质量最大的分词方案。

- 状态定义： $dp[i]$ 表示前 i 个字符的最大分词质量和
- 转移方程：

$$dp[i] = \max_{0 \leq j < i} \{dp[j] + quality(y[j+1..i])\}$$

- 边界条件： $dp[0] = 0$ （空字符串质量为 0）
- 计算顺序： i 从 1 到 n 递增
- 时间复杂度： $O(n^2)$

```
int maxQualitySegmentation(string y, function<int(string)>
quality) {
    int n = y.length();
    vector<int> dp(n + 1, INT_MIN / 2); // 避免溢出
    dp[0] = 0;

    for (int i = 1; i <= n; i++) {
        // 枚举所有切分点 j
        for (int j = 0; j < i; j++) {
            string sub = y.substr(j, i - j);
            int q = quality(sub);
            dp[i] = max(dp[i], dp[j] + q);
        }
    }

    return dp[n];
}

// 示例：使用哈希表预存词汇质量
unordered_map<string, int> dictQuality = {
    {"a", 1}, {"ab", 3}, {"abc", 4},
    {"b", 1}, {"bc", 2}, {"c", 1}
};

int solve(string y) {
    return maxQualitySegmentation(y, [](string s) {
        return dictQuality.count(s) ? dictQuality[s] : INT_MIN / 2;
    });
}
```

1. 建一维数组， $dp[i]$ 表示前 i 个字符切分后的最大质量
2. 从前往后：尝试让最后一段从 $j+1$ 到 i ，看是否是词典里的词
3. 如果是词，就用“前 j 个字符的最大质量”加上“这个词的质量”
4. 最终 $dp[n]$ 即为答案

6. 带权活动选择问题

给定 n 个活动，每个活动 $a_i = (s_i, f_i, v_i)$ ，其中 s_i 为开始时间， f_i 为结束时间， v_i 为权重。选择若干不重叠的活动，使权重和最大。

- 预处理：按结束时间 f_i 升序排序
- 前驱查找： $prev[i]$ 表示最后一个结束时间 $\leq s_i$ 的活动（二分查找）
- 状态定义： $dp[i]$ 表示前 i 个活动的最大权重和
- 转移方程：

$$dp[i] = \max(dp[i-1], dp[prev[i]] + v_i)$$

- 时间复杂度： $O(n \log n)$

```
struct Activity {
    int s, f, v; // 开始时间, 结束时间, 权重
};

int weightedActivitySelection(vector<Activity>& acts) {
    int n = acts.size();

    // 按结束时间排序
    sort(acts.begin(), acts.end(),
        [](const Activity& a, const Activity& b) {
            return a.f < b.f;
        });

    // 预处理 prev 数组（二分查找）
    vector<int> prev(n + 1, 0);
    for (int i = 1; i <= n; i++) {
        int l = 0, r = i - 1;
        while (l < r) {
            int mid = (l + r + 1) / 2;
            if (acts[mid-1].f <= acts[i-1].s)
                l = mid;
            else
                r = mid - 1;
        }
        prev[i] = l;
    }

    // DP
    vector<int> dp(n + 1, 0);
    for (int i = 1; i <= n; i++) {
        dp[i] = max(dp[i-1], dp[prev[i]] + acts[i-1].v);
    }

    return dp[n];
}
```

1. 将所有活动按结束时间从小到大排序
2. 用二分查找找出每个活动“最后一个不与它冲突的活动”
3. 建一维数组， $dp[i]$ 表示前 i 个活动的最大权重
4. 比较“不选它”和“选它”（ $dp[prev[i]]$ 加权重），取较大值

7. 最大子数组问题（分治法）

给定 n 个整数（有正有负）的数组 A ，设计 $O(n \log n)$ 算法，找出和最大的非空连续子数组。

- 分治思想：
 - 分解：将数组分为左半 L 和右半 R
 - 解决：递归求左、右最大子数组
 - 合并：计算跨越中点的最大子数组，取三者最大值
- 跨越中点的计算：从中点向左记录最大后缀和，向右记录最大前缀和，相加得跨越最大值
- 递推公式：

$$T(n) = 2T(n/2) + O(n) \Rightarrow T(n) = O(n \log n)$$

```
// 跨越中点的最大子数组和
int maxCrossing(vector<int>& A, int l, int mid, int r) {
    // 向左找最大后缀和
    int leftSum = INT_MIN, sum = 0;
    for (int i = mid; i >= l; i--) {
        sum += A[i];
        leftSum = max(leftSum, sum);
    }

    // 向右找最大前缀和
    int rightSum = INT_MIN;
    sum = 0;
    for (int i = mid + 1; i <= r; i++) {
        sum += A[i];
        rightSum = max(rightSum, sum);
    }

    return leftSum + rightSum;
}

// 分治主函数
int maxSubarray(vector<int>& A, int l, int r) {
    if (l == r) return A[l]; // 基础情况

    int mid = (l + r) / 2;

    // 三者取最大
    int leftMax = maxSubarray(A, l, mid);
    int rightMax = maxSubarray(A, mid + 1, r);
    int crossMax = maxCrossing(A, l, mid, r);

    return max({leftMax, rightMax, crossMax});
}

int maxSubarray(vector<int>& A) {
    return maxSubarray(A, 0, A.size() - 1);
}
```

- 如果只有一个元素，直接返回
- 从中间切开，分别递归求左右两半的最大子数组
- 再算跨越中点的：中点往左最大后缀加中点往右最大前缀
- 在”左边最大””右边最大””跨越最大”三者中取最大

8. 最长非降子序列（LIS）

给定长度为 n 的序列 A ，求最长非降子序列（LIS）的长度（非降指 $a_i \leq a_j$ 对 $i < j$ 成立，子序列不要求连续）。

方法 1：基础 DP ($O(n^2)$)

- 状态定义： $dp[i]$ 表示以 $A[i]$ 结尾的最长非降子序列长度
- 转移方程：若 $A[j] \leq A[i]$ 则 $dp[i] = \max(dp[i], dp[j] + 1)$
- 边界条件： $dp[i] = 1$ （至少包含自己）

方法 2：贪心 + 二分 ($O(n \log n)$)

- 核心思想：维护数组 $tail[len]$ ，表示长度为 len 的非降子序列的最小末尾元素
- 操作：若 $A[i] > tail[len]$ 则扩展，否则二分查找替换第一个 $> A[i]$ 的位置

```
// 方法1: 基础DP O(n^2)
int LIS_basic(vector<int>& A) {
    int n = A.size();
    vector<int> dp(n, 1);

    for (int i = 1; i < n; i++) {
        for (int j = 0; j < i; j++) {
            if (A[j] <= A[i]) {
                dp[i] = max(dp[i], dp[j] + 1);
            }
        }
    }

    return *max_element(dp.begin(), dp.end());
}

// 方法2: 贪心+二分 O(n log n)
int LIS_optimized(vector<int>& A) {
    vector<int> tail; // tail[len] = 长度为len+1的LIS最小末尾

    for (int x : A) {
        if (tail.empty() || x > tail.back()) {
            tail.push_back(x);
        } else {
            // 二分查找第一个> x的位置
            auto it = lower_bound(tail.begin(), tail.end(), x);
            *it = x;
        }
    }

    return tail.size();
}
```

基础 DP 步骤：

- 建一维数组， $dp[i]$ 表示以位置 i 结尾的最长非降子序列长度
- 对每个位置，往左找所有比它小的位置 j ，更新 $dp[i]$
- 取所有 $dp[i]$ 中的最大值

优化版步骤：

- 维护一个数组，记录”长度为 x 的非降子序列末尾的最小可能值”
- 遇到新数：如果比末尾大就接在后面，否则用二分查找替换第一个比它大的位置
- 最终数组长度就是答案，时间 $O(n \log n)$

9. 矩阵连乘问题

给定 n 个矩阵 $M_1, M_2, \dots, M_n$, 其中 M_i 维数为 $r_{i-1} \times r_i$ 。矩阵乘法满足结合律, 不同的括号化方式导致不同的乘法次数。求使乘法运算次数最少的最优括号化方案。

- 状态定义: $m[i][j]$ 表示矩阵 M_i 到 M_j 连乘的最小乘法次数
- 转移方程:

$$m[i][j] = \min_{i \leq k < j} \{m[i][k] + m[k+1][j] + r_{i-1} \times r_k \times r_j\}$$

- 边界条件: $m[i][i] = 0$
- 计算顺序: 按区间长度 $x = j - i + 1$ 从 1 到 n 递增
- 时间复杂度: $O(n^3)$

```
int matrixChainOrder(vector<int>& r) {
    int n = r.size() - 1; // n个矩阵
    vector<vector<int>> m(n + 1, vector<int>(n + 1, 0));
    // 按区间长度递增
    for (int x = 2; x <= n; x++) { // x = j-i+1
        for (int i = 1; i <= n - x + 1; i++) {
            int j = i + x - 1;
            m[i][j] = INT_MAX;

            for (int k = i; k < j; k++) {
                int cost = m[i][k] + m[k+1][j] + r[i-1] * r[k] * r[j];
                if (cost < m[i][j]) {
                    m[i][j] = cost;
                }
            }
        }
    }
    return m[1][n];
}

// 带最优解构造版本
pair<int, string> matrixChainWithSolution(vector<int>& r) {
    int n = r.size() - 1;
    vector<vector<int>> m(n + 1, vector<int>(n + 1, 0));
    vector<vector<int>> s(n + 1, vector<int>(n + 1, 0)); // 记录切分点

    for (int x = 2; x <= n; x++) {
        for (int i = 1; i <= n - x + 1; i++) {
            int j = i + x - 1;
            m[i][j] = INT_MAX;

            for (int k = i; k < j; k++) {
                int cost = m[i][k] + m[k+1][j] + r[i-1] * r[k] * r[j];
                if (cost < m[i][j]) {
                    m[i][j] = cost;
                    s[i][j] = k;
                }
            }
        }
    }
    return {m[1][n], "可通过s数组回溯构造括号化方案"};
}
```

1. 建二维表, $m[i][j]$ 表示从第 i 个矩阵乘到第 j 个矩阵的最少乘法次数
2. 按区间长度从小到大算
3. 枚举分割点 k : 代价等于” 左半边” 加” 右半边” 加” 最后一次乘法”
4. 取所有 k 中代价最小的那个

10. 钢条切割问题

给定长度为 n 英寸的钢条和价格表 p_i ($i = 1 \sim n$), 钢条可切割为任意段, 求切割后总收益最大。例如: $n = 4, p = [1, 5, 8, 9]$ 时, 最优方案为切成 2 段长度 2, 收益 $5 + 5 = 10$ 。

- 状态定义: $r[n]$ 表示长度为 n 的钢条的最大收益
- 转移方程:

$$r[n] = \max_{1 \leq i \leq n} \{p[i] + r[n - i]\}$$

- 边界条件: $r[0] = 0$
- 时间复杂度: $O(n^2)$

```
// 方法1: 自底向上DP
int rodCutting(vector<int>& p, int n) {
    vector<int> r(n + 1, 0);
    r[0] = 0;

    for (int i = 1; i <= n; i++) {
        int maxVal = INT_MIN;
        for (int j = 1; j <= i; j++) {
            maxVal = max(maxVal, p[j-1] + r[i - j]);
        }
        r[i] = maxVal;
    }

    return r[n];
}

// 方法2: 带解构造
pair<int, vector<int>> rodCuttingWithSolution(vector<int>& p, int n) {
    vector<int> r(n + 1, 0);
    vector<int> s(n + 1, 0); // s[i] = 长度i的最优第一刀切割位置

    for (int i = 1; i <= n; i++) {
        int maxVal = INT_MIN;
        for (int j = 1; j <= i; j++) {
            if (p[j-1] + r[i - j] > maxVal) {
                maxVal = p[j-1] + r[i - j];
                s[i] = j;
            }
        }
        r[i] = maxVal;
    }

    // 回溯构造解
    vector<int> cuts;
    int temp = n; // 保存原始n值
    while (temp > 0) {
        cuts.push_back(s[temp]);
        temp -= s[temp];
    }

    return {r[n], cuts};
}
```

1. 建一维数组, $r[i]$ 表示长度为 i 的钢条最大收益
2. 从长度 1 到 n : 对每个长度 i , 枚举第一刀切多长 j
3. 第一刀切 j , 卖 $p[j]$ 的钱, 剩下的长度 $i-j$ 继续求最优
4. 取所有 j 中收益最大的填入 $r[i]$

11. 最长公共子序列（LCS）

给定两个序列 $X = (x_1, x_2, \dots, x_m)$ 和 $Y = (y_1, y_2, \dots, y_n)$ ，求其最长公共子序列的长度及具体序列（子序列不要求连续）。例如： $X = \text{"ABCBADAB"}$ ， $Y = \text{"BDCABA"}$ ，LCS 为 "BCAB" 或 "BDAB" ，长度为 4。

- 状态定义： $dp[i][j]$ 表示 $X[1..i]$ 与 $Y[1..j]$ 的 LCS 长度
- 转移方程：若 $X[i] = Y[j]$ 则 $dp[i][j] = dp[i-1][j-1] + 1$ ，否则 $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$
- 边界条件： $dp[0][j] = 0$ ， $dp[i][0] = 0$
- 时间复杂度： $O(mn)$

```
// 计算LCS长度
int LCSLength(string X, string Y) {
    int m = X.length(), n = Y.length();
    vector<vector<int>>> dp(m + 1, vector<int>(n + 1, 0));

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (X[i-1] == Y[j-1]) {
                dp[i][j] = dp[i-1][j-1] + 1;
            } else {
                dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
            }
        }
    }

    return dp[m][n];
}

// 回溯构造LCS序列
string LCS(string X, string Y) {
    int m = X.length(), n = Y.length();
    vector<vector<int>>> dp(m + 1, vector<int>(n + 1, 0));

    // 填充DP表
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (X[i-1] == Y[j-1]) {
                dp[i][j] = dp[i-1][j-1] + 1;
            } else {
                dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
            }
        }
    }

    // 回溯构造LCS
    string lcs = "";
    int i = m, j = n;
    while (i > 0 && j > 0) {
        if (X[i-1] == Y[j-1]) {
            lcs = X[i-1] + lcs;
            i--; j--;
        } else if (dp[i-1][j] > dp[i][j-1]) {
            i--;
        } else {
            j--;
        }
    }

    return lcs;
}
```

1. 建二维表， $dp[i][j]$ 表示两个串前 i 和前 j 个字符的 LCS 长度
2. 字符相同就等于左上角加 1，不相同就取左边和上边较大的
3. 第一行和第一列全为 0
4. 右下角就是 LCS 长度

12. 最优二叉搜索树

给定 n 个有序关键字 $k_1 < k_2 < \dots < k_n$ ，每个关键字 k_i 的成功搜索概率为 p_i ，失败搜索概率为 $q_0, q_1, \dots, q_n$ 。构造使平均搜索长度最小的二叉搜索树。

- 状态定义： $c[i][j]$ 表示关键字 $k_{i+1} \sim k_j$ 的最优 BST 的平均搜索长度， $w[i][j]$ 表示区间 $[i, j]$ 的总权值
- 转移方程：

$$c[i][j] = w[i][j] + \min_{i < k \leq j} \{c[i][k-1] + c[k][j]\}$$

- 边界条件： $c[i][i] = 0$ ， $w[i][i] = q_i$
- 时间复杂度： $O(n^3)$

```
double optimalBST(vector<double>& p, vector<double>& q, int n) {
    // c[i][j]: 区间[i,j]的最小代价
    // w[i][j]: 区间[i,j]的总权值
    // root[i][j]: 区间[i,j]的根节点（用于构造BST）
    vector<vector<double>>> c(n + 2, vector<double>(n + 2, 0));
    vector<vector<double>>> w(n + 2, vector<double>(n + 2, 0));
    vector<vector<int>>> root(n + 2, vector<int>(n + 2, 0));

    // 初始化：区间长度为0
    for (int i = 1; i <= n + 1; i++) {
        w[i][i-1] = q[i-1];
        c[i][i-1] = 0;
    }

    // 按区间长度递增
    for (int x = 1; x <= n; x++) {
        for (int i = 1; i <= n - x + 1; i++) {
            int j = i + x - 1;
            w[i][j] = w[i][j-1] + p[j] + q[j];
            c[i][j] = 1e300; // 相当于double类型的无穷大

            // 枚举根节点k
            for (int k = i; k <= j; k++) {
                double cost = c[i][k-1] + c[k+1][j] + w[i][j];
                if (cost < c[i][j]) {
                    c[i][j] = cost;
                    root[i][j] = k;
                }
            }
        }
    }

    return c[1][n]; // 最小平均搜索长度
}
```

1. 建二维表， $c[i][j]$ 表示用关键字 $i+1$ 到 j 构建 BST 的最小平均搜索长度
2. 按区间长度从小到大算
3. 枚举哪个关键字当根
4. 取所有根中代价最小的

优化方法：利用 Knuth 优化（四边形不等式），将枚举根的范围从 $[i, j]$ 缩小至 $[root[i][j-1], root[i+1][j]]$ ，时间复杂度可降至 $O(n^2)$ 。

13. DAG 最安全路径问题

给定有向无环图 $G = (V, E)$ ，边 e_i 的事故概率为 p_i 。路径安全性定义为路径上所有边概率的乘积。求从起点 s 到终点 t 的最安全路径（安全性最大的路径）。

- 状态定义： $dp[u]$ 表示从起点 s 到节点 u 的最大传输安全性
- 转移方程：对所有出边 (u, v) ,

$$dp[v] = \max(dp[v], dp[u] \times p_{uv})$$

- 边界条件： $dp[s] = 1$ ，其余节点 $dp[u] = 0$
- 拓扑排序：按拓扑序遍历节点，保证 u 的所有前驱已处理
- 时间复杂度： $O(V + E)$

```
struct Edge {
    int to;
    double prob; // 安全概率
};

double safestPathDAG(int n, vector<vector<Edge>>& adj, int s, int t) {
    vector<double> dp(n, 0.0);
    dp[s] = 1.0;

    // 拓扑排序 (Kahn算法)
    vector<int> inDegree(n, 0);
    for (int u = 0; u < n; u++) {
        for (Edge& e : adj[u]) {
            inDegree[e.to]++;
        }
    }

    queue<int> q;
    for (int u = 0; u < n; u++) {
        if (inDegree[u] == 0) q.push(u);
    }

    // 按拓扑序DP
    while (!q.empty()) {
        int u = q.front(); q.pop();

        for (Edge& e : adj[u]) {
            // 松弛操作
            if (dp[u] * e.prob > dp[e.to]) {
                dp[e.to] = dp[u] * e.prob;
            }

            if (--inDegree[e.to] == 0) {
                q.push(e.to);
            }
        }
    }

    return dp[t]; // 返回最大安全性
}
```

1. 先做拓扑排序：统计入度，入度为 0 的点入队处理
2. 建一维数组 dp ，起点设为 1，其余为 0
3. 按拓扑序处理：对出边用”当前 dp 值乘概率”更新终点

14. 环形房间化学品存放问题

给定 n 个首尾相连的环形房间，每个房间容量为 w_i （杂物房间容量为 0）。约束：化学品不能存放在相邻房间。求最多能存放的化学品总容量。

线性序列 DP（打家劫舍）：

- 状态定义： $dp[i]$ 表示前 i 个房间的最大容量
- 转移方程： $dp[i] = \max(dp[i - 1], dp[i - 2] + w_i)$
- 边界条件： $dp[0] = 0, dp[1] = w_1$

环形转线性：首尾相邻，拆分为两个线性问题（不选第 1 个或不选第 n 个）

```
// 线性序列DP (打家劫舍)
int houseRobberLinear(vector<int>& w, int start, int end) {
    int n = end - start + 1;
    if (n == 0) return 0;
    if (n == 1) return w[start];

    int prev2 = 0; // dp[i-2]
    int prev1 = w[start]; // dp[i-1]

    for (int i = start + 1; i <= end; i++) {
        int curr = max(prev1, prev2 + w[i]);
        prev2 = prev1;
        prev1 = curr;
    }

    return prev1;
}

// 环形房间问题主函数
int houseRobberCircular(vector<int>& w) {
    int n = w.size();
    if (n == 0) return 0;
    if (n == 1) return w[0];
    if (n == 2) return max(w[0], w[1]);

    // 情况1: 不选第1个房间, 考虑2~n
    int case1 = houseRobberLinear(w, 1, n - 1);

    // 情况2: 不选第n个房间, 考虑1~n-1
    int case2 = houseRobberLinear(w, 0, n - 2);

    return max(case1, case2);
}
```

1. 环形拆成两个线性问题：不选第 1 个、不选第 n 个
2. 线性 DP： $dp[i]$ 表示前 i 个房间的最大容量
3. 比较”不选 i ”和”选 i ”
4. 取两种情况的最大值

15. 受限最短路径（带钱数约束）

给定无向图 $G = (V, E)$ ，边 e_{uv} 长度为 l_{uv} ，顶点 u 花费为 $c[u]$ 。从 s 出发到 t ，初始钱数为 M 。每经过顶点 u 需支付 $c[u]$ （剩余钱数 $\geq c[u]$ 才能通过），求 s 到 t 的最短路径长度。

- 状态扩展: $dist[u][m]$ 表示到达顶点 u 、剩余钱数为 m 时的最短路径长度

- 状态转移: 对边 (u, v) ,

$$dist[v][m - c[v]] = \min(dist[v][m - c[v]], dist[u][m] + l_{uv})$$

- 初始化: $dist[s][M] = 0$ ，其余为 ∞

- 优先队列: 按路径长度从小到大处理状态

```
struct Edge {
    int to, len;
};

struct State {
    int u, money;
    int dist;
    bool operator>(const State& other) const {
        return dist > other.dist;
    }
};

int constrainedShortestPath(int n, int M, int s, int t,
    vector<int>& cost,
    vector<vector<Edge>>& adj) {
    // dist[u][m] = 到达u、剩余m元的最短距离
    vector<vector<int>> dist(n, vector<int>(M + 1, INT_MAX / 2));
    dist[s][M] = 0;

    // 优先队列（小根堆）
    priority_queue<State, vector<State>, greater<State>> pq;
    pq.push({s, M, 0});

    while (!pq.empty()) {
        State cur = pq.top(); pq.pop();

        int u = cur.u, m = cur.money, d = cur.dist;

        if (d > dist[u][m]) continue; // 已优化
        if (u == t) return d; // 到达终点

        for (Edge& e : adj[u]) {
            int v = e.to;
            int newMoney = m - cost[v];

            if (newMoney >= 0 && d + e.len < dist[v][newMoney]) {
                dist[v][newMoney] = d + e.len;
                pq.push({v, newMoney, dist[v][newMoney]});
            }
        }
    }

    return -1; // 不可达
}
```

1. 状态扩展: $dist[u][m]$ 表示“到达 u 且剩余钱为 m 时的最短距离”
2. 起点 $dist[s][M]=0$ ，其余为无穷大
3. 用优先队列按距离处理，对出边松弛操作

16. 完全占满背包（恰好装满）

给定 n 个物品，物品 i 的大小为 w_i 、价值为 v_i ，背包容量为 C 。要求选择一组物品，恰好占满背包容量（选中物品大小和恰好为 C ），且总价值最大。

- 状态定义: $dp[j]$ 表示恰好占满容量 j 时的最大价值

- 初始化调整: $dp[0] = 0$, $dp[j] = -\infty$ ($j > 0$)

- 转移方程: 若 $dp[j - w_i] \neq -\infty$, $dp[j] = \max(dp[j], dp[j - w_i] + v_i)$

- 最终结果: $dp[C]$

```
int knapsackExact(vector<int>& w, vector<int>& v, int C) {
    int n = w.size();
    const int NEG_INF = INT_MIN / 2;

    vector<int> dp(C + 1, NEG_INF);
    dp[0] = 0; // 容量0恰好被占满

    for (int i = 0; i < n; i++) {
        // 逆序遍历（避免重复选择）
        for (int j = C; j >= w[i]; j--) {
            if (dp[j - w[i]] != NEG_INF) {
                dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
            }
        }
    }

    return dp[C] == NEG_INF ? -1 : dp[C]; // -1表示无解
}

// 测试
int main() {
    vector<int> w = {2, 3, 4};
    vector<int> v = {3, 4, 5};
    int C = 5;

    int result = knapsackExact(w, v, C);
    if (result != -1) {
        cout << "恰好装满的最大价值: " << result << endl;
    } else {
        cout << "无法恰好装满背包" << endl;
    }

    return 0;
}
```

1. 初始化: $dp[0]=0$ ，其余都是负无穷大
2. 对每个物品，容量从大到小遍历
3. 只有当 $dp[j-\text{重量}]$ 不是负无穷时，才能更新 $dp[j]$
4. 保证只有“恰好装满”的状态之间才能转移

17. 数组最大子段的个数（O(n)）

设 $A[1..n]$ 是长度为 n 的数组，存放可重复的整数。最大子段指其和不少于任何子段和的子段。要求设计 $O(n)$ 算法，求最大子段的个数。

- 状态定义: $currSum$ 当前和, $currCount$ 当前个数, $globalSum$ 全局最大, $globalCount$ 全局个数
- 转移规则: 若 $A[i] > currSum + A[i]$ 则新子段; 若 $A[i] < currSum + A[i]$ 则延续; 若相等则个数相加
- 全局更新: 若 $currSum > globalSum$ 则更新; 若相等则累加个数
- 时间复杂度: $O(n)$

```
int maxSubarrayCount(vector<int>& A) {
    int n = A.size();
    if (n == 0) return 0;

    long long currSum = A[0];
    long long currCount = 1;
    long long globalSum = A[0];
    long long globalCount = 1;

    for (int i = 1; i < n; i++) {
        long long sumExtend = currSum + A[i];
        long long sumNew = A[i];

        if (sumNew > sumExtend) {
            // 新子段更优
            currSum = sumNew;
            currCount = 1;
        } else if (sumNew < sumExtend) {
            // 延续更优
            currSum = sumExtend;
        } else {
            // 两者相等，个数相加
            currSum = sumExtend;
            currCount += currCount;
        }

        // 更新全局最大值
        if (currSum > globalSum) {
            globalSum = currSum;
            globalCount = currCount;
        } else if (currSum == globalSum) {
            globalCount += currCount;
        }
    }

    return globalCount;
}
```

1. 维护”当前和”、”当前个数”、”全局最大”、”全局个数”
2. 比较”重新开始”和”延续前面”哪个更大
3. 相等则累加个数

18. 最少插入字符使字符串成为回文

给定字符串 S ，设计算法在 S 中插入最少的字符，使其成为回文字符串。例如: $S = \text{”ab”}$ ，插入 1 个字符得”aba”或”bab”。

原因: 回文中”已有的最长回文部分”等价于 S 与 S^{rev} 的 LCS，剩余部分需插入对应字符补全回文。

- 核心结论: 最少插入字符数 = $|S| - \text{LCS 长度}$
- 方法: LCS 为原串 S 与其反转串 S^{rev} 的最长公共子序列
- 算法步骤: 求 S^{rev} ，求 LCS，计算 $|S| - \text{LCS}$
- 时间复杂度: $O(n^2)$

```
int minInsertionsToPalindrome(string S) {
    int n = S.length();

    // 构造反转字符串
    string S_rev = S;
    reverse(S_rev.begin(), S_rev.end());

    // 求LCS长度
    vector<vector<int>> dp(n + 1, vector<int>(n + 1, 0));

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            if (S[i-1] == S_rev[j-1]) {
                dp[i][j] = dp[i-1][j-1] + 1;
            } else {
                dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
            }
        }
    }

    int lcsLen = dp[n][n];
    return n - lcsLen; // 最少插入字符数
}

// 空间优化版本（O(n)空间）
int minInsertionsToPalindromeOptimized(string S) {
    int n = S.length();
    string S_rev = S;
    reverse(S_rev.begin(), S_rev.end());

    vector<int> dp(n + 1, 0);

    for (int i = 1; i <= n; i++) {
        int prev = 0; // dp[i-1][j-1]
        for (int j = 1; j <= n; j++) {
            int temp = dp[j];
            if (S[i-1] == S_rev[j-1]) {
                dp[j] = prev + 1;
            } else {
                dp[j] = max(dp[j], dp[j-1]);
            }
            prev = temp;
        }
    }

    return n - dp[n];
}
```

1. 要插入的字符数等于”原串长度减去原串的最长回文子序列长度”
2. 原串的最长回文子序列等价于”原串”和”反转串”的 LCS
3. 先求反转串，再用 LCS 算法求两者的最长公共子序列
4. 原串长度减去 LCS 长度就是答案

19. 二维矩阵的最大子矩阵和

给定 $m \times n$ 的二维整数矩阵 A (含正、负、零)，找到和最大的子矩阵，并输出其和。

二维转一维：枚举所有可能的行范围，压缩列后用 Kadane 算法

- **枚举行范围：**遍历所有上边界 top 和下边界 $bottom$
- **压缩列：**对每个行范围 $[top, bottom]$ ，计算每列的和得到 $colSum[]$
- **Kadane 算法：**对 $colSum[]$ 求最大子数组
- **时间复杂度：** $O(m^2 \cdot n)$

```
// Kadane 算法 (返回最大子数组和)
int kadane(vector<int>& arr) {
    int maxSum = arr[0];
    int currSum = arr[0];

    for (int i = 1; i < arr.size(); i++) {
        currSum = max(arr[i], currSum + arr[i]);
        maxSum = max(maxSum, currSum);
    }

    return maxSum;
}

// 最大子矩阵和
int maxSubmatrixSum(vector<vector<int>>& A) {
    int m = A.size();
    if (m == 0) return 0;
    int n = A[0].size();

    // 前缀和预处理
    vector<vector<int>> prefix(m + 1, vector<int>(n, 0));
    for (int i = 1; i <= m; i++) {
        for (int j = 0; j < n; j++) {
            prefix[i][j] = prefix[i-1][j] + A[i-1][j];
        }
    }

    int maxSum = INT_MIN;

    // 枚举所有行范围
    for (int top = 0; top < m; top++) {
        for (int bottom = top; bottom < m; bottom++) {
            // 压缩列: 计算 colSum[]
            vector<int> colSum(n);
            for (int j = 0; j < n; j++) {
                colSum[j] = prefix[bottom+1][j] - prefix[top][j];
            }

            // Kadane 算法求最大子数组
            int currMax = kadane(colSum);
            maxSum = max(maxSum, currMax);
        }
    }

    return maxSum;
}
```

1. 先算前缀和，方便 $O(1)$ 求任意行范围内某列的和
2. 枚举所有可能的行边界，对每个行范围，压缩成一维数组
3. 用 Kadane 算法求最大子数组

复杂度分析

- 前缀和预处理: $O(mn)$
- 枚举行范围: $O(m^2)$
- 每次 Kadane: $O(n)$
- 总时间复杂度: $O(m^2 \cdot n)$ (若 $m \approx n$, 则为 $O(n^3)$)